

Work Group 7 Project Report

Pi-Squared: Real-Time Communication Between Two Raspberry Pi Devices

Abhinav Ramalingam

Vanja Caroline Borowski

Zixuan He

21st October 2024

Abstract

This project presents the development of a chat application between two Raspberry Pi devices using the MQTT protocol. The application aims to provide real-time messaging over a local network by leveraging the lightweight paho-mqtt library for MQTT-based communication. The project involves setting up a local MQTT broker using Docker with Eclipse Mosquitto, adding message logging capabilities and client interfaces for the command line and browser. The Raspberry Pi devices act as clients in the system, enabling seamless exchange of messages. Python is used as the primary programming language. The project underscores the use of modern IoT technologies in creating an efficient, and user-friendly communication platform between edge devices.

Contents

1	Introduction	1
1.1	Purpose and Goals	1
1.2	Report Outline	2
2	Background	3
2.1	Major Concepts Used in the Project	3
2.1.1	Raspberry Pi	3
2.1.2	Message Queuing Telemetry Transport	3
2.1.3	Docker	4
2.2	Related Work	5
3	Methodology	7
3.1	Message Flow	7
3.2	Message Logging in MQTT	9
3.3	Ethical Considerations	9
3.3.1	Privacy and Data Security	9
3.3.2	Security Vulnerabilities	10
4	Implementation	11
4.1	Software Tools	11
4.2	Hardware Tools	11
4.3	Broker Implementation	12
4.4	Client Implementation	13
5	Contributions	15
6	Conclusion	17
6.1	Key Takeaways	17
6.2	Key Findings	17
6.3	Future Enhancements and Considerations	18
	References	19

List of Figures

3.1	Message from device 2 to device 1	8
3.2	Message from device 1 to device 2	8
4.1	Backend implementation schematics	13
4.2	Two Raspberry Pis communicating via the CLI client	14
4.3	Two Raspberry Pis communicating via the GUI client	14

Chapter 1

Introduction

Internet of Things, shortly termed IoT, is a significant technical advancement that combines electronics, sensors, and network connectivity with physical objects—like gadgets, cars, and buildings—to enable smooth data transfer and remote control. The idea was first brought to life by Kevin Ashton in 1999, with an initial emphasis on RFID technology. Since then, it has developed into a sophisticated system incorporating several technologies, including RFID, wireless sensor networks, and cloud computing. IoT is the most recent stage in the development of the Internet, moving away from document-centric and commerce-driven networks and towards a network where machines and other linked items are essential components (Gokhale et al., 2018).

The ability to facilitate intelligent communication and operations among a wide range of devices and services makes IoT increasingly relevant in determining the direction of technology in the future. IoT makes it easier for commonplace objects to connect and exchange data by giving them distinct identities, from basic ones like hairbrushes and tires to complex smart devices. Efficient data collection, processing, and exchange are made possible by integrating disparate entities, including those with integrated sensors and different protocols. As a result, with little to no human involvement, IoT facilitates complicated, autonomous operations and decision-making processes. Its influence is substantial since it fosters innovation and improves efficiency in many businesses and daily life by building a more intelligent and connected technology environment (Abdul-Qawy et al., 2015).

Thus, the importance of IoT can be summarized in two main aspects:

1. IoT allows for *enhanced automation* in real-time
2. IoT generates *large amounts of data* that can be analyzed for insights
 - More on this: Data is available in everyday items like wearables, smart refrigerators, and thermostats. Using the gathered data, one can optimize daily operations, forecast energy consumption, and improve public health, which supports more informed decision-making.

1.1 Purpose and Goals

One of the key aspects of the Raspberry Pi that can be explored is its networking capabilities (Raspberry Pi, 2015). This enables seamless communication and data

transfer between devices, making it an ideal platform for IoT applications. The project's main purpose is to develop an end-to-end, fully functional, and secure Chat Application that works between two Raspberry Pis. This involves building a reliable communication protocol such as Message Queuing Telemetry Transport (MQTT), implementing secure communication, and designing user-friendly input and output functionality for the two devices. The overall group goals of the project are listed below:

1. **Configuration of the Raspberry Pi:** Establish a reliable configuration on the Pi, which is both accessible and reproducible for further projects.
2. **Implementation of the Chat Back-end:** Design, Implement and Test the Chat server and the Chat client on the Pi, demonstrating the effectiveness of the Raspberry Pi in communication.
3. **Designing the display system for the chat application:** Implement an intuitive and user-friendly interface for participants to interact with the Pi.
4. **Integration of the Project Components:** Assembling all the implemented components into a fully functional Chat application, emphasizing on the modularity of the design.
5. **Final Documentation:** Enable knowledge transfer by developing a comprehensive documentation about the project to share the insights gained with future IoT enthusiasts.

The project serves as a proof of concept for the idea that the Raspberry Pi can effectively utilize MQTT and other communication protocols to implement secure communication systems. Conversely, this project also explores the capabilities of the Pi itself to be able to execute complex communication protocols to enable communication.

1.2 Report Outline

This section presents how the rest of the thesis is structured. Chapter 2 introduces the **Background** knowledge related to Raspberry Pi and MQTT protocol, addressing other technology which is to be used in the functionality of the project. Chapter 3 outlines the systematic **Methodology** used to design and implement the application, detailing the tools used and project sub-goals. Chapter 4 describes the overall **Implementation**, including the configuration of the devices, user interface design, and networking integration. Chapter 5 details the specific roles and **Contributions** of each group member in the project, highlighting their effort which influenced the overall implementation as outlined in the team contract. And lastly, Chapter 6 summarises the key takeaways and highlights future possibilities for improvement.

Chapter 2

Background

In this chapter, an exploration of the various concepts used in the project's implementation is presented. This encompasses both the hardware and software aspects of the project, include the underlying Raspberry Pi and related configuration, the communication protocols used to execute the process, and other important concepts incorporated so that the overall integration of the project is reliable.

2.1 Major Concepts Used in the Project

2.1.1 Raspberry Pi

The Raspberry Pi, introduced by the Raspberry Pi Foundation in 2012, is a compact, single-board computer designed to provide an accessible entry point for individuals interested in computing and programming. Its small size, comparable to that of a credit card, houses all essential processing components on a single circuit board, including a GPU, CPU, USB ports, RAM, and an SD card slot that typically holds the operating system and other necessary files. This design simplicity makes the Raspberry Pi an ideal platform for learners, as it is not only affordable but also operates on Linux, allowing users to experiment with a variety of programming tasks. (Whitmore, 2022).

Moreover, Raspberry Pi's integration of IoT technology facilitates communication with other devices on the same network, enabling users to automate processes and create smart applications with relatively minimal coding. Its versatility extends to various applications, such as powering digital signage for advertisers, displaying real-time information on dashboards, and serving as a low-cost solution for monitoring a wide range of variables. Different models of the Raspberry Pi have emerged over the years, each offering varying levels of technological capabilities, making it suitable for both novice users and more advanced projects

2.1.2 Message Queuing Telemetry Transport

Message Queuing Telemetry Transport, shortly referred to as MQTT, is a lightweight messaging protocol designed for resource-constrained devices operating on low-bandwidth, high-latency, or unreliable networks that render it particularly suitable for IoT applications. Its lightweight nature, characterized by minimal overhead and small packet sizes, allows efficient communication among devices with limited processing power and memory. Reliability is another key feature, as MQTT supports

various Quality of Service (QoS) levels, session awareness, and persistent connections, ensuring message delivery even under challenging network conditions (EMQX, 2024).

Security is paramount in IoT, and MQTT addresses this with Transport Layer Security (TLS) and Secure Sockets Layer (SSL) encryption, along with authentication mechanisms to protect sensitive data. Its bi-directional communication model enables seamless device interaction, facilitating effective data exchange without direct coupling, thus simplifying device integration and scaling. MQTT also maintains continuous, stateful sessions, allowing the broker to remember client subscriptions and undelivered messages, which mitigates data loss during intermittent connectivity.

The protocol's inherent scalability makes it exceptionally suited for large-scale IoT systems, directly aligning with the project's need for robust communication among multiple Raspberry Pis. Its extensive language support facilitates seamless integration across various programming environments, ensuring flexibility in development. The MQTT ecosystem is composed of clients—devices or applications utilizing the MQTT client library—and brokers, which efficiently manage client connections and message routing. This design supports numerous simultaneous connections and high message throughput, all while employing the publish-subscribe pattern that decouples publishers from subscribers, significantly enhancing overall system efficiency for the chat application.

2.1.3 Docker

Docker is an open-standard platform that simplifies the building, exporting, and running of applications through a command-line interface and background daemon, addressing common software deployment challenges. It allows developers to separate applications from their underlying infrastructure, enabling quicker software delivery via Docker containers, which encapsulate all necessary dependencies for independent execution. This isolation and security feature facilitates the simultaneous operation of multiple containers on a single host machine. (Bhatia et al., 2017).

Docker was first developed to run on the Linux Operating System (OS) but has expanded to support non-Linux platforms like MacOS and Windows, including versions for cloud services like AWS and Microsoft Azure. Before Docker's introduction, deploying software across different environments was labor-intensive and time-consuming, requiring significant management of virtual machines. Docker streamlines the deployment process by enabling users to pull images from repositories to run with minimal resource consumption and independence from the host OS.

The advantages of Docker containers are particularly beneficial for the project, as they are free and open-source, providing a cost-effective solution. Docker containers achieve ultra-consistency due to their lower resource requirements, enabling rapid deployment in seconds, which is crucial for the iterative development process. Their portability as lightweight applications allows for easy migration across different environments. Additionally, Docker ensures isolation between containers, enhancing security by preventing interference and resource sharing, thereby protecting data integrity within each container, which is vital for maintaining reliable communication in the chat application.

2.2 Related Work

A lot of interesting and related work/projects are developed in the field of IoT (especially Raspberry Pi) and/or MQTT protocols, discussion which could be relevant to the project.

The authors of Anne et al. (2022) developed a mobile chat app using the protocol, focusing on its implementation through the Eclipse MQTT server and Android Studio, with the Mosquitto broker facilitating the publish/subscribe model. The performance of this model was compared with existing chat applications, and the features of MQTT were evaluated against other application layer protocols. Testing was conducted on Android devices with versions higher than 4.0, demonstrating that MQTT is particularly effective in networks with constrained devices, limited bandwidth, or unreliable connections.

Similarly, the authors of Hussein and Shujaa (2020) designed and implemented an ambulance tracking and messaging system to address road congestion issues that affect emergency healthcare services. Using the MQTT protocol, a central monitoring unit tracks the ambulance, which is equipped with a Raspberry Pi-based intelligent embedded system. The ambulance communicates with other vehicles or road traffic via the CoAP protocol for direct device-to-device communication. The system includes a hands-free voice chat feature, utilizing Google API for speech recognition and text-to-speech conversion. Messages are encrypted with a stream cipher, using One Time Pad (OTP) and DNA computing, with a unique key for each message generated by a linear feedback shift register (LFSR).

The insights gained from the related work highlight the importance of designing reliable communication protocols, especially in IoT environments, which serves as inspiration for our own work. Our project seeks to explore the capabilities of the Raspberry Pi in conjunction with communication protocols by implementing a chat application. The project not only aims to establish a foundational understanding of Raspberry Pi's potential, but also to create a robust base application that can facilitate future improvements on the same.

Chapter 3

Methodology

This section will focus on the proposed method used to implement the chat application and the ethical considerations. The initial setup will involve two Raspberry Pis representing two clients in a one-to-one chat scenario.

3.1 Message Flow

The proposed method will use the MQTT protocol send and receive messages. The lightweight protocol follows a publisher/subscriber model which consists of MQTT-Clients and MQTT-Brokers. In this scenario, each Raspberry Pi will function as a MQTT-Client.

A Broker serves as the central entity responsible for handling message distribution. Messages are routed based on topics which the broker distributes to all subscribed clients. The broker can either be a separate device or on the same device as a client. For this project the broker will be implemented as a separate entity to simulate a cloud-based messaging application. A docker container will be used to implement the broker, which offers several advantages. A containerised solution will ensure that the broker operates in an isolated environment which eliminates interference with other services or dependencies installed on the host system. While this could be achieved with a virtual machine, the ease of use and portability of the eclipse-mosquitto broker is preferred.

Clients communicate exclusively with the broker to send and receive messages. To receive messages, the client subscribes to one or more topics by establishing a connection with the broker, as illustrated in Fig. 3.1. In this scenario each topic corresponds to a specific chat session. To send a message, the client publishes it to the broker, which then distributes the message to all clients subscribed to the respective topic (Fig. 3.2)¹. Since there is no restriction to the number of subscribed clients, this method allows multiple clients to participate in a single chat, thus allowing the realisation of group chats with the same method.

¹The split into two diagrams might suggest that the topics “Messages” in Fig. 3.2 and Fig. 3.1 are disparate topics, this is not the case and was done to increase readability.

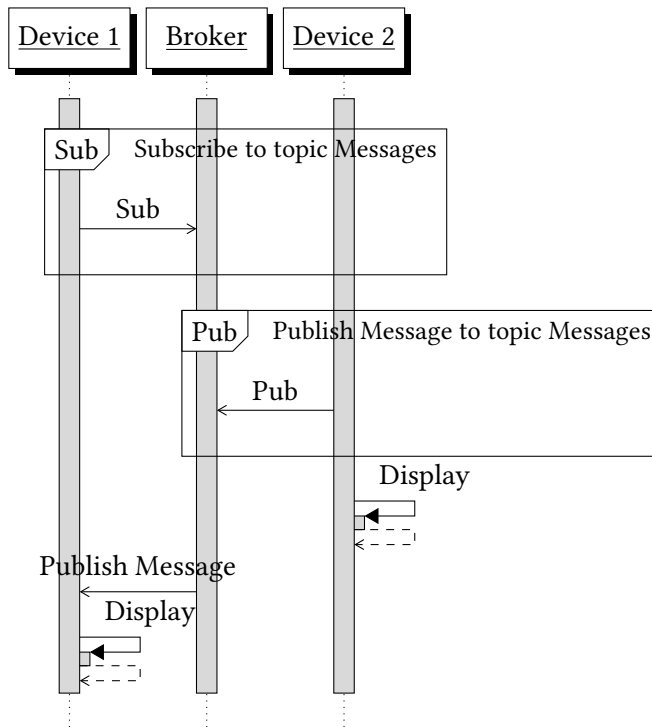


Figure 3.1: Message from device 2 to device 1

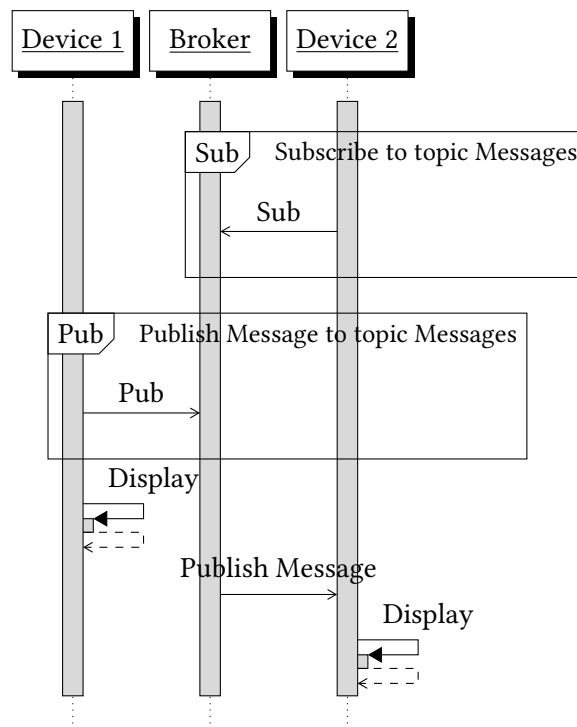


Figure 3.2: Message from device 1 to device 2

To ensure that the subscribing clients reliably receive the messages for their subscribed topics, the Quality of Service (QoS) feature of MQTT will be utilised. When connecting to the broker, each client specifies the QoS level for message delivery.

A value of 0 indicates that messages are sent without guaranteeing delivery, as no re-transmission will be attempted, regardless of whether the client receives the message. For a value of 1, the message must be received by the recipient at least once, duplicates may occur and for a value of 2, the message must be delivered exactly once, duplicate messages will be dismissed (OASIS Message Queuing Telemetry Transport (MQTT) TC, 2019). As a chat application relies on messages being received, the application will use a minimum QoS Level of 1, to ensure a correct message flow, at the cost of an increased communication overhead.

3.2 Message Logging in MQTT

The MQTT protocol is primarily a messaging protocol and has no specification for message logging. While there are existing broker implementations that provide a database connection, we decided to implement this ourselves (see Chapter 4). The two main components in MQTT communication are one broker and 1 to n clients, with clients being able to subscribe to one or more topics. Topics are structured hierarchically and with variable depth, e.g. `"/topic/subtopic/subsubtopic"`.

Special characters for topics are the wildcards `"+"` and `"#"`. While `"+"` can be used to match everything for the depth of one level, `"#"` matches to everything in all remaining levels (OASIS Message Queuing Telemetry Transport (MQTT) TC, 2019). To provide logging capabilities in our chat application, we will use MQTTs wildcard capabilities and add a special client which uses the `"#"` wildcard to subscribe to all chats. A connected database will be used as a message storage.

3.3 Ethical Considerations

This section addresses the ethical concerns surrounding privacy, data security, and vulnerabilities in communication between Raspberry Pi devices, focusing on safeguarding sensitive data, ensuring secure transmissions, and preventing unauthorized access.

3.3.1 Privacy and Data Security

Deploying a chat application between two Raspberry Pi devices introduces potential privacy and data security concerns, as user messages could be susceptible to interception or unauthorized or third-party access during transmission.

Mitigating Privacy Risks

To address these privacy concerns, the integration of robust encryption methods, such as asymmetric encryption (e.g. RSA), can be implemented to ensure that all messages are transmitted securely on the application level, and remain confidential. It ensures that only authorized recipients can decrypt the messages, protecting data integrity throughout the communication process. Additionally, leveraging well-established secure communication protocols would reinforce privacy by securing the communication channel against potential eavesdropping. In fact, MQTT includes built-in support for Transport Layer Security (TLS) and Secure Sockets Layer (SSL), ensuring encrypted communication and safeguarding data integrity during transmission.

3.3.2 Security Vulnerabilities

Exposing devices to unprotected networks poses significant risks, including the potential for exploitation through malware, unauthorized access, or more sophisticated threats like Man-in-the-Middle attacks. Such vulnerabilities could undermine the integrity of the communication and compromise both devices involved.

Addressing Security Risks

Beyond encryption, implementing secure user authentication mechanisms is critical to ensuring that only legitimate devices participate in the communication. Thus, we can mitigate unauthorized access attempts, securing the communication from intrusion by establishing authentication to the broker. Combined with the payload encryption addressed in 3.3.1 this will ensure, that messages will only be read by the intended recipients.

Chapter 4

Implementation

This section details the key components and processes of our project implementation, explaining the reasoning behind our choice of specific technologies to develop the chat application. We emphasize the integration of Raspberry Pi and communication protocols to achieve our project objectives effectively.

4.1 Software Tools

In the process of developing a chat application project using two Raspberry Pis, we primarily utilized Python, paho-mqtt, Docker, and Visual Studio Code. Specifically, this section provides a detailed explanation of why each tool was selected and its importance in the project.

- **Programming Language:** The entire project was developed using Python due to its versatility and extensive library support, particularly for network communication and interfacing with hardware like Raspberry Pis. Python's readability and ease of use made it ideal for developing and maintaining the chat application.
- **MQTT Library:** For handling MQTT-based communication, we primarily used paho-mqtt. This library was selected due to its simplicity, robustness, and well-documented support for MQTT, a lightweight messaging protocol suitable for IoT applications like ours.
- **Running the MQTT Broker:** To run the MQTT broker locally, we utilized Docker to containerize the MQTT broker. Docker was chosen because it simplifies deployment and testing by allowing the broker to be run in an isolated environment that can be easily reproduced on different devices or systems without additional setup complexity.

4.2 Hardware Tools

The hardware setup will consist of two Raspberry Pi devices, each acting as a client in the chat application. The MQTT broker can either be run on a third device or a PC/laptop on the same network. These devices were chosen for their cost-effectiveness and versatility in handling lightweight applications like chatting program.

- **Raspberry Pi Model:** The project used Raspberry Pi 3 Model B, equipped with a quad-core Cortex-A72 processor and 8GB of RAM, which provided enough computing power to handle the chat application and MQTT communication.
- **Storage:** Each Raspberry Pi has a 16GB microSD card for the operating system and software storage, sufficient for the application's needs.
- **Peripheral Setup:** The idea is to connect an HDMI screen to display chat messages, along with a keyboard and mouse for initial setup and configuration. Alternatively, one can use SSH to connect to the Pi, allowing access via a terminal interface.
- **Network:** The Raspberry Pi devices must be connected to the same local network to facilitate communication between the MQTT clients and the broker (In future work, we can explore configuring a public IP address for the MQTT broker and potentially setting up a domain name by purchasing our own domain).

4.3 Broker Implementation

The MQTT broker is implemented as a Docker container running Eclipse Mosquitto. This choice was motivated by the need for a flexible, replicable environment for running the broker. By containerizing the broker, we ensured it can be easily deployed on any compatible system without dependency conflicts.

As mentioned in Chapter 3 MQTT does not provide a way to preserve messages other than for re-submission to the client for certain Quality of Service settings, which does not suffice as permanent storage. To counter this, we added a special client and a database. These components run in their own containers and can be deployed either on the same host as the broker or on separate hosts.

For this project we decided to use a single host setup for the entire backend. Since all three components are hosted in Docker containers, we used Docker compose to simplify the deployment and to ensure that the containers always start in the correct order.

As seen in Fig. 4.1 the MQTT-broker publishes all messages to the logging client. We implemented this by having the logging client subscribe to all topics. This ensures that the logger will receive all messages sent via the broker.

The third component, which holds the database, exposes two API endpoints. `/log` is used by the logging client to store messages in the database. `/getChat` can be queried by clients to get the chat history for a specific chat. Upon message receipt the logging client uses the log endpoint to store the message. Depending on the client implementation, `getChat` will be queried when the user enters a chat.

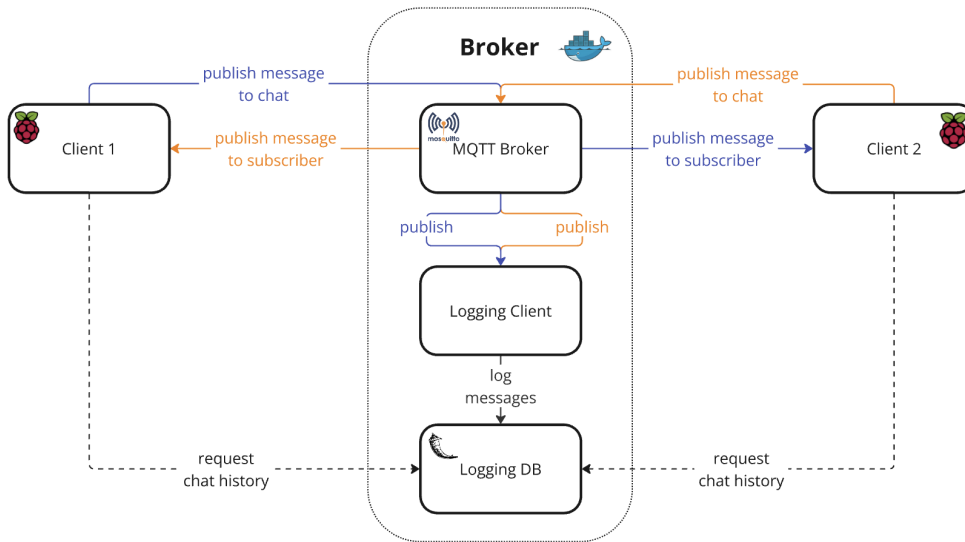


Figure 4.1: Backend implementation schematics

4.4 Client Implementation

The chat application on the Raspberry Pi devices is developed following a client-server architecture. Especially, each Raspberry Pi acted as a client and they communicated with each other via an MQTT broker. The client implementation focused on setting up the Raspberry Pis to end and receive messages, were displayed in a CLI and GUI. The specific content is described in this section:

1. **MQTT Communication:** Each Raspberry Pi is configured as an MQTT client using the paho-MQTT Python library. The clients connect to the central broker and exchange messages through the publish-subscribe model.
2. **Input and Output Interfaces:**
 - (a) **Command-Line Interface (CLI):** This version of the client displays messages via the command line. When a user enters a chat, the client queries all past messages from the database and displays them accordingly. Users type their messages and send them with enter, which transmits them to the broker via MQTT. The broker then delivers the messages to all subscribed clients, where they are displayed on the CLI (as seen in Fig. 4.2).
 - (b) **Graphical User Interface (GUI):** A more visual client version which utilizes Python and Tkinter to create a simple yet functional GUI. This GUI allows users to enter and view messages in a user-friendly way. The GUI provides an input box for typing messages and a display area for showing received messages.
3. **Real-time Messaging:** The core function of the client is the real-time transmission and display of chat messages. The messaging system was tested to ensure minimal latency, ensuring a smooth conversation between the two devices. Performance tests showed the system met real-time requirements, with negligible delays.

4. **Network Setup:** Both Raspberry Pi clients were connected to the same local network to enable seamless communication with the MQTT broker. The network environment was tested for stability and reliability during message transmission.

The above is a detailed introduction to the experimental process of this project. The output in Fig. 4.3 shows the two raspberry Pi devices communicating with each other via a Graphical User Interface.

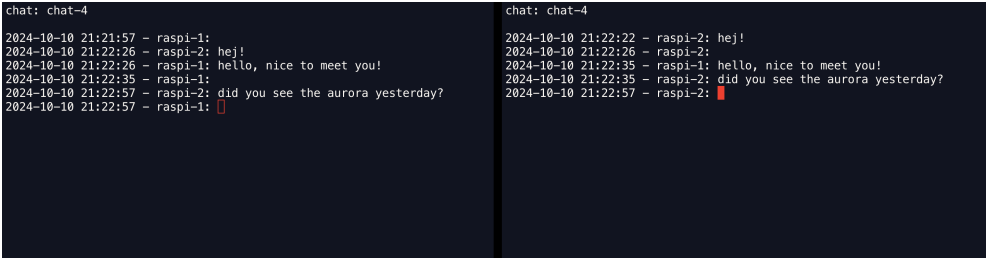
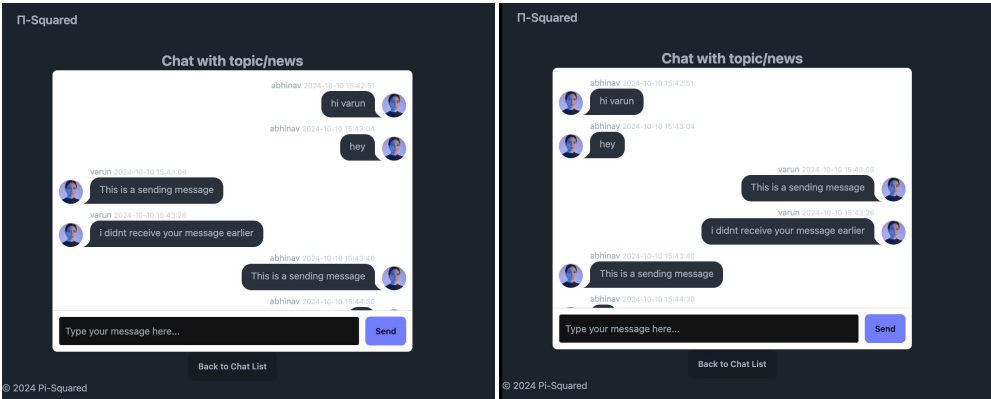


Figure 4.2: Two Raspberry Pis communicating via the CLI client



(a) client 1

(b) client 2

Figure 4.3: Two Raspberry Pis communicating via the GUI client

Chapter 5

Contributions

This section highlights the contributions made by each group member in the project, encompassing both the report contributions and parts of the technical implementation they undertook.

Abhinav Ramalingam contributed significantly to the project by writing the *Introduction* and *Background* chapters of the report. In the implementation phase, he focused on the *client functionality implementation using Flask* and the *user interface implementation using Jinja2 and JavaScript*. His efforts were crucial in establishing a solid foundation for the chat application.

Zixuan He played a vital role by authoring the *Implementation* and *Conclusion* chapters of the report. Her technical expertise was instrumental in the *Raspberry Pi configuration and integration of the components*. Zixuan's contributions ensured that all components functioned cohesively, allowing for a seamless user experience.

Vanja Caroline Borowski was responsible for the *Methodology* chapter and parts of the *Implementation* chapter, where she outlined the project's approach and procedures. In terms of implementation, she implemented the *backend using MQTT, Python and Docker* for the broker and *Python and Flask* for the logging components. She was also responsible for the *client command line interface (CLI) implementation using Python*. Her work was essential in creating a robust and efficient communication system for the chat application.

Chapter 6

Conclusion

This project successfully demonstrated the feasibility of establishing a secure and reliable real-time communication system using two Raspberry Pi devices via an MQTT-based chat application. The main goals of this project is to develop an effective messaging system with real-time performance, intuitive user interfaces, and reliable network communication.

When starting this project, we focused on two main takeaways. During the experimentation and analysis phase, we made three key findings. Finally, we proposed several improvement measures for the project.

6.1 Key Takeaways

1. **Real-Time Messaging:** The combination of Raspberry Pi, Python, and MQTT resulted in an efficient and scalable solution for IoT-based communication, demonstrating the suitability of these tools for building real-time applications.
2. **Practical Implementation:** The project's success in utilizing Docker for easy deployment and management of the MQTT broker highlighted Docker's effectiveness in simplifying the setup and ensuring a replicable development environment.

6.2 Key Findings

1. **Core Functionality:** The project successfully met its primary goal of enabling real-time chat between two Raspberry Pi devices. The system leveraged the MQTT protocol to ensure low-latency communication, which was crucial for maintaining smooth interactions between the clients. Both the command-line interface (CLI) and graphical user interface (GUI) provided efficient platforms for sending and receiving messages.
2. **Efficient and Stable Communication:** The use of MQTT proved to be highly efficient for handling lightweight and real-time data transmission. Extensive testing demonstrated that the system performed reliably with minimal delays, even under varied network conditions, confirming the stability and responsiveness of the chat application.
3. **User-Friendly Interfaces:** The development of both CLI and GUI implementations provided flexibility in user interaction. While the CLI offered a func-

tional interface suitable for restricted environments, the GUI provided a second user experience by offering a more graphic or appealing way to manage communications, both aligning with the project's goal of creating a user-friendly input and output system.

6.3 Future Enhancements and Considerations

1. **Security:** Implementing additional authentication and encryption for message transmission and API access should be added to further enhance the confidentiality and security of communication and addressing potential privacy concerns and ethical considerations outlined in Section 3.3 in future deployments.
2. **GUI Improvements:** Enhancing the GUI with modern libraries and introducing advanced features such as multimedia support, message replies and reactions/emojis, and other innovative user interactions, providing a more comprehensive communication platform.
3. **Broker Location:** The project currently utilises the Raspberry Pis as simple clients. In the future we want to explore shifting the location of the backend to one or more Raspberry Pis, which would showcase the IoT capabilities of our project even more. The modularity of the backend allows the exploration of multiple scenarios in which a Pi can either host one or more backend components. It would also be possible to use a Pi as both a broker and a client.

In conclusion, this project not only demonstrated the technical feasibility of using Raspberry Pi devices with MQTT for real-time communication but also contributed to understanding how IoT devices can be effectively utilized in practical applications. The system's modularity, stability, and user-centric design make it a strong foundation for further development, particularly in educational or IoT-based communication systems.

References

- Abdul-Qawy, A. S., Pramod, P., Magesh, E., and Srinivasulu, T. (2015). The internet of things (iot): An overview. *International Journal of Engineering Research and Applications*, 5(12):71–82.
- Anne, L., Nandan, T., Kunj, M., Kumar, S. A., Mahesh, G., and Sangeetha, R. (2022). Mqtt-based android chat application for iot. *SN Computer Science*, 3(5):402.
- Bhatia, G., Choudhary, A., and Gupta, V. (2017). The road to docker: a survey. *International Journal of Advanced Research in Computer Science*, 8(8):83–87.
- EMQX (2024). Mqtt protocol explained: Ultimate guide for iot beginners. <https://www.emqx.com/en/blog/the-easiest-guide-to-getting-started-with-mqtt>. (Accessed on 2024-09-16).
- Gokhale, P., Bhat, O., and Bhat, S. (2018). Introduction to iot. *International Advanced Research Journal in Science, Engineering and Technology*, 5(1):41–44.
- Hussein, N. A. and Shujaa, M. I. (2020). Secure vehicle to vehicle voice chat based mqtt and coap internet of things protocol. *Indonesian Journal of Electrical Engineering and Computer Science*, 19(1):526–534.
- OASIS Message Queuing Telemetry Transport (MQTT) TC (2019). MQTT specification. <https://mqtt.org/mqtt-specification/>. (Accessed on 2024-09-23).
- Raspberry Pi (2015). Buy a Raspberry Pi 3 Model B. <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>. (Accessed on 2024-09-22).
- Whitmore, C. (2022). What is a raspberry pi and how does it work? <https://nordvpn.com/blog/what-is-raspberry-pi/#:~:text=A%20Raspberry%20Pi%20has%20everything,and%20potentially%20some%20more%20files>. (Accessed on 2024-09-17).